

MOD300 Anvendt Python programmering og modellering

Enrico Riccardi¹

Department of Mathematics and Physics, University of Stavanger (UiS).¹

30.10.2025



© 2025, Enrico Riccardi. Released under CC Attribution 4.0 license

A linear model

Considering a univariate case, we have:

$$q = f(x)$$

which relates the **independent variable** x to the **true dependent variable** q .

Assuming a linear model

$$q = \beta_0 + \beta_1 x$$

where β_i are the arbitrary selected coefficients.

Model set-up

For a given x we do not know the true response q , only the measurement y_i for experiment i .

We have that:

$$y_i = q_i + \epsilon_i$$

NOTE: do not proceed if you do not fully understand this equation.

which is

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

Discussion point

Does ϵ_i matter? And why so?

Estimated model parameters

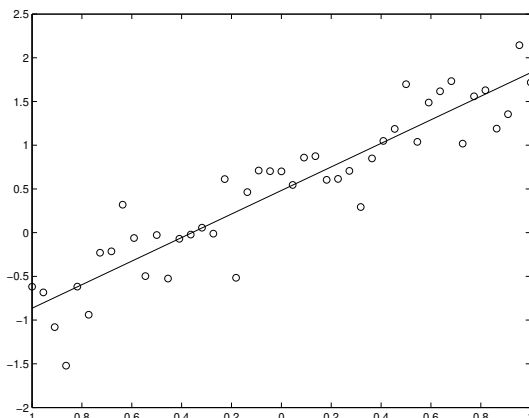
The model parameters β_0, β_1 are unknown, but they can be **estimated**. To distinguish estimates from true model parameters we call them b_0, b_1 . These estimates are calculated such that the model

$$\hat{y} = b_0 + b_1 x$$

fits the n different experimental observations as well as possible.

Linear model example

We would like to find the TRUTH (What it is?)



Python Example

```
import numpy as np
import matplotlib.pyplot as plt

def generate_linear_data(n_random_points, noise=16):
    x = np.random.rand(n_random_points) * 10

    # Make 'perfect' data
    true_slope, true_intercept = 2, 5
    y = true_slope * x + true_intercept

    # Add noise
    y += np.random.randn(n_random_points)*noise

    return x, y, true_slope, true_intercept

# Use the function to generate data
x, y, true_slope, true_intercept = generate_linear_data(
    n_random_points=166,
    noise=3)

# Plot all
plt.plot(x, true_slope*x + true_intercept,
         color='red', label='Truth Line')
plt.scatter(x, y, color='blue', label='Data Points')
plt.show()
```

Estimation of linear regression parameters

For the 1-dimensional problem, we have

$$\hat{y} = b_0 + b_1 x$$

where $\hat{y}$ is the estimated y-value from the approximate model that has been generated from a set of measurements (x_i, y_i) . We aim to find the b_i parameters such that the regression line fits the observed data as well as possible.

This means we want to minimise the residuals

$$e_i = y_i - \hat{y}_i$$

Let's recaps

Before to continue, let's make sure to have all the main elements clear:

- ❶ Splitting the data in dependent and independent variables
- ❷ Assumption of a linear model between them
- ❸ Recognise the difference between the truth and the estimation
- ❹ Aiming to *minimize* the residuals

Discussion

What happen when the sum of residual is 0 ?

What happens where the data is heavily correlated?

Regression

Skipping the math (but you are more than welcome to try), here are the results:

$$\begin{aligned} b_0 &= \bar{y} - b_1 \bar{x} \\ b_1 &= \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \end{aligned}$$

Straightforward derivation becomes very cumbersome for multiple variables. Thus, a different approach must be used. Yet, it is important to understand that there is an analytical solution (even if not all the time).

Estimation of linear regression parameters

- Cannot sum e_i values since they might be positive and negative and thus cancel
- Could use e.g. $\sum_{i=1}^n |e_i|$, but is mathematically more difficult to handle
- Residual "smallness" measured by $\sum_{i=1}^n e_i^2$.

Thus, we find the linear regression coefficients by *minimising*

$$R = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

How?

Regression!

Regression

To minimize the sum of the square residuals, we can try to solve the following equations:

$$\begin{aligned} \frac{\partial R}{\partial b_0} &= 0 \\ \frac{\partial R}{\partial b_1} &= 0 \end{aligned}$$

where:

$$\begin{aligned} R &= \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \\ &= \sum_{i=1}^n (y_i - (b_0 + b_1 x_i))^2 = \sum_{i=1}^n u_i^2 \end{aligned}$$

Linear model(s)

Does a linear model mean only straight lines (or hyperplanes in general)?

That answer to this is *no*. different. In general for a model f to be defined linear, it has to be linear with respect to the unknown parameters $\beta_0, \dots, \beta_n$. The general linear model is

$$q = \beta_0 + \beta_1 f_1(x_1) + \beta_2 f_2(x_2) + \dots + \beta_n f_n(x_n)$$

where $f_i(x_i)$ may be non-linear functions. It is the **form** of the equation which makes it linear, i.e. that $f_i(x_i)$ does not depend on the parameters β_i .

Linear model(s)

Consider the following model example - is it linear?

$$q = \beta_0 + \beta_1 x_1^2 + \beta_2 x_2^{-1} + \beta_3 \log x_3$$

The answer is yes because by simple substitution it is possible to convert this formula into a linear form.

With $h_1 = x_1^2$, $h_2 = x_2^{-1}$ and $h_3 = \log x_3$, then we can formulate the new model:

$$q = \beta_0 + \beta_1 h_1 + \beta_2 h_2 + \beta_3 h_3$$

which is in the standard linear form.

Curvilinear models

A special class of linear models which we will investigate later are those which are expressed in terms of *polynomials* (here in only 1D):

$$q = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_m x^m = \sum_{i=0}^m \beta_i x^i$$

For n-D case there are a wide range of interaction terms and combinations, that can still be converted to standard linear form.

Such models are sometimes referred to as **curvilinear** instead of non-linear

Nonlinear models

But there are many other models that cannot be substituted to such a form. For instance:

$$q = \beta_0 + \log(x - \beta_1)$$

No substitution can transform this equation to the linear form.

That is the case for all the model that:

$$q \sim f(x, \beta)$$

Another example is:

$$f(x, \beta) = \frac{x\beta}{x + \beta}$$

Many variable equation

The general equation would look like:

$$\hat{y} = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_n x_n = \sum_{j=0}^m x_{ij} b_j$$

where we will have to solve all the $n + 1$ equations (called the **normal equations**) of the form:

$$\frac{\partial R}{\partial b_j} = 0 \quad \forall j \in [0, n]$$

Is there a way for us to simplify this?

We can use vector and matrix algebra.

Regression

To minimize the sum of the square residuals, we can try to solve the following equations:

$$\begin{aligned} \frac{\partial R}{\partial b_0} &= 0 \\ \frac{\partial R}{\partial b_1} &= 0 \end{aligned}$$

where:

$$\begin{aligned} R &= \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \\ &= \sum_{i=1}^n (y_i - (b_0 + b_1 x_i))^2 = \sum_{i=1}^n u_i^2 \end{aligned}$$

Regression

Skipping the math (but you are more than welcome to try), here are the results:

$$\begin{aligned} b_0 &= \bar{y} - b_1 \bar{x} \\ b_1 &= \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \end{aligned}$$

Straightforward derivation becomes very cumbersome for multiple variables. Thus, a different approach must be used. Yet, it is important to understand that there is an analytical solution (even if not all the time).

Many variable equation

The general equation would look like:

$$\hat{y} = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_n x_n = \sum_{j=0}^m x_{ij} b_j$$

where we will have to solve all the $n + 1$ equations (called the **normal equations**) of the form:

$$\frac{\partial R}{\partial b_j} = 0 \quad \forall j \in [0, n]$$

Is there a way for us to simplify this?

We can use vector and matrix algebra.

Regression via Matrix operation

Remember that

$$R = \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

then we can define the vector $\mathbf{e}$:

$$\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$$

thus

$$\mathbf{e}^T = [(y_1 - \hat{y}_1) \ (y_2 - \hat{y}_2) \cdots \ (y_N - \hat{y}_N)]$$

and can then write

$$R = \mathbf{e}^T \mathbf{e}$$

Regression via Matrix operation

Remember that

$$R = \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

From the following equation:

$$\hat{y}_i = b_0 + \sum_{j=1}^m x_{ij} b_j = \sum_{j=0}^m x_{ij} b_j$$

then we can define the vector $\mathbf{e}$:

$$\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$$

thus

$$\mathbf{e}^T = [(y_1 - \hat{y}_1) \ (y_2 - \hat{y}_2) \cdots \ (y_N - \hat{y}_N)]$$

and can then write

$$R = \mathbf{e}^T \mathbf{e}$$

where $x_{i0} = 1$
we make the matrix equation:

$$\hat{\mathbf{y}} = \mathbf{X} \mathbf{b}$$

where the first column in $\mathbf{X}$
consists of ones only.

Residual

$$\begin{aligned} R &= \mathbf{e}^T \mathbf{e} \\ &= (\mathbf{y} - \hat{\mathbf{y}})^T (\mathbf{y} - \hat{\mathbf{y}}) \\ &= (\mathbf{y} - \mathbf{X} \mathbf{b})^T (\mathbf{y} - \mathbf{X} \mathbf{b}) \\ &= (\mathbf{y}^T - \mathbf{b}^T \mathbf{X}^T) (\mathbf{y} - \mathbf{X} \mathbf{b}) \\ &= \mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X} \mathbf{b} - \mathbf{b}^T \mathbf{X}^T \mathbf{y} \\ &\quad + \mathbf{b}^T \mathbf{X}^T \mathbf{X} \mathbf{b} \end{aligned}$$

All the parts of this equation are
scalar values. This means
e.g. that

$$\mathbf{y}^T \mathbf{X} \mathbf{b} = \mathbf{b}^T \mathbf{X}^T \mathbf{y}$$

This gives

$$R = \mathbf{y}^T \mathbf{y} - 2 \mathbf{y}^T \mathbf{X} \mathbf{b} + \mathbf{b}^T \mathbf{X}^T \mathbf{X} \mathbf{b}$$

Residual

But how can we now compute $\frac{\partial R}{\partial b_j}$ more efficiently in matrix form?

Vector differentiation ! Let

$$\mathbf{y} = \mathbf{a}^T \mathbf{x} = a_1 x_1 + \dots + a_n x_n$$

If

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y}{\partial x_1} \\ \frac{\partial y}{\partial x_2} \\ \vdots \\ \frac{\partial y}{\partial x_n} \end{bmatrix} = \begin{bmatrix} a_1 \\ \vdots \\ a_n \end{bmatrix} = \mathbf{a}$$

and $\mathbf{y} = \mathbf{x}^T \mathbf{a}$, then:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{a}$$

General solution

In general, when $\mathbf{y} = \mathbf{x}^T \mathbf{A} \mathbf{x}$, then

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = 2 \mathbf{A} \mathbf{x}$$

if $\mathbf{A}$ is symmetric
(check *Matrix calculus* for more properties)

We can use this to compute

$$\frac{\partial R}{\partial \mathbf{b}}$$

General solution

We have from above:

$$R = y^T y - 2y^T Xb + b^T X^T X b$$

Vector differentiation gives

$$\frac{\partial R}{\partial b} = 0 - 2X^T y + 2X^T X b = 0$$

Solving this for b we get:

$$\begin{aligned} X^T X b &= X^T y \\ (X^T X)^{-1} X^T X b &= (X^T X)^{-1} X^T y \\ b &= (X^T X)^{-1} X^T y \end{aligned}$$

Multiple linear regression

Previous equation make the solution of MLR rather obvious!

When we have a matrix of y -variables Y :

$$B = (X^T X)^{-1} X^T Y$$

in the equation:

$$Y = XB.$$

These equations give us the **multiple linear regression** (MLR) solution.

Loss function

One of the main core concepts behind machine learning is the **loss function**.

In mathematical optimization and decision theory, a loss function or cost function (sometimes also called an error function) is a function that maps an event or values of one or more variables onto a real number intuitively representing some "cost" associated with the event. [Wiki]

Loss function in Machine Learning

It quantifies the difference between the predicted outputs of a machine learning algorithm and the actual target values.

Loss function aims

It provides a set of core quantifications/possibilities:

- 1 Performance measurement: it provides the metric of the prediction performances
- 2 Direction for improvement: it allows the identification of convergent solutions
- 3 Balancing bias and variance: it allows to account for artefact in the sampling phase
- 4 Influencing model behaviour: it allows to bridge data driven methods with mathematics/physics

Most common loss function in machine learning

Most popular entries:

- 1 Mean Square Error (regression): Fast computations, good convergence.
- 2 Mean Absolute Error (regression): No focus on the outliers, poor convergence.
- 3 Entropy Loss (classification): measures the difference between the model probability distribution outcomes and the predicted values

Losses

Mean absolute error, L1 loss

$$MAE = \frac{1}{n} \sum_{i=1}^n (|y_i - \bar{y}|)$$

Mean square error, L2 loss

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

Log Loss for binary classification

$$Log_Loss = -[y \log(f(x)) + (1 - y) \log(1 - f(x))]$$

MSE in Vanilla Python

```
def mean_squared_error(observed, predicted):
    """ Compute the mean squared error.

    Params:
    -----
    observed : list,
        The observed values.
    predicted : list,
        The predicted values.

    Output:
    -----
    sys[0] : float,
        Mean squared error.

    """
    if len(observed) != len(predicted):
        raise ValueError(
            "The lengths of input lists are" +
            f"not equal {len(observed)} {len(predicted)}."
        )

    # Initialize the sum of squared errors
    sum_squared_error = 0
```

MSE in Vanilla Python

```
sum_squared_error = 0

# Loop through all observations
for obs, pred in zip(observed, predicted):
    # Calculate the square difference, and add it to the sum
    sum_squared_error += (obs - pred) ** 2

# Calculate the mean squared error
mse = sum_squared_error / len(observed)

return mse
```

MSE in Numpy Python

```
import numpy as np

def mean_squared_error(observed, predicted):
    observed_np = np.array(observed)
    predicted_np = np.array(predicted)
    mse = np.mean((observed_np - predicted_np) ** 2)
    return mse
```

Regularization

Conventionally, MSE is used as loss function:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

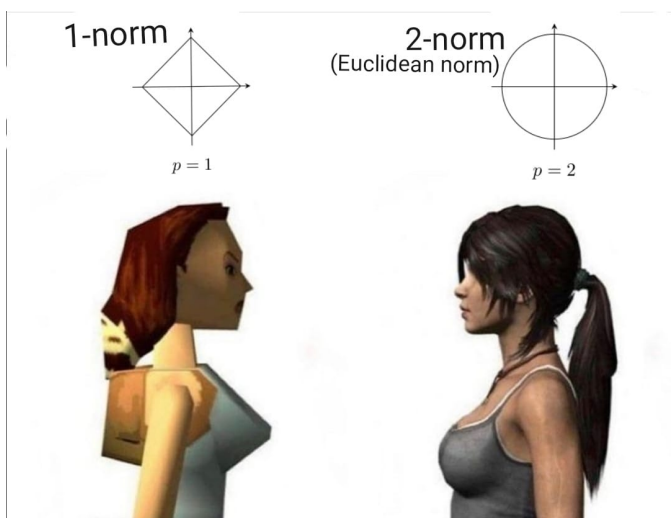
We might want to force our model not to fit the data so well, i.e. we *penalise* the model.

$$L1_loss_regularized = MSE + \lambda \sum_{i=1}^m |b_i|$$

$$L2_loss_regularized = MSE + \lambda \sum_{i=1}^m b_i^2$$

$$Elastic\ Net = MSE + \lambda_1 \sum_{i=1}^m |b_i| + \lambda_2 \sum_{i=1}^m b_i^2$$

Norms



Outcomes



Regularization

λ is the regularisation strength. The larger value it has, the more the target function is "wrong". For $\lambda = 0$, no regularisation is imposed.

Forcing a model to be "wrong" might sound rather counter-intuitive. Furthermore, we have introduced one more parameter that has not much meaning?!

Bias - variance trade off

$$Error = bias^2 + variance + statistical\ error$$

Bias

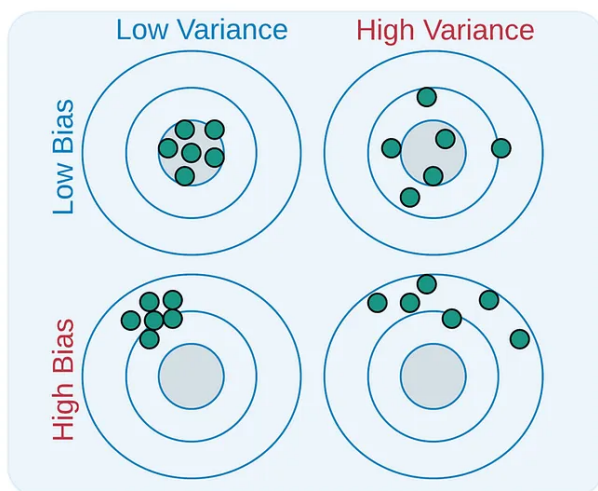
Bias is the difference of the average value of predictions (q) from the true relationship function (f):

$$bias[q(x)] = \mathbb{E}[q(x)] - f(x)$$

Variance is the expectation of the squared deviation of $q(x)$ from its expected value $\mathbb{E}[q(x)]$.

$$var[q(x)] = \mathbb{E}[(q(x) - \mathbb{E}[q(x)])^2]$$

Bias-Variance



Bias-Variance

